

Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования  
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии  
Департамент цифровых, робототехнических систем и электроники

**ОТЧЕТ**  
**ПО ЛАБОРАТОРНОЙ РАБОТЕ №6**  
**дисциплины «Искусственный интеллект и машинное обучение»**  
**Вариант 7**

Выполнила:  
Еремина Татьяна Евгеньевна  
2 курс, группа ИВТ-б-о-24-1,  
09.03.01 «Информатика и  
вычислительная техника», очная  
форма обучения

---

(подпись)

Проверил:  
Воронкин Р.А., доцент департамента  
цифровых, робототехнических систем  
и электроники института  
перспективной инженерии

---

(подпись)

Отчет защищен с оценкой \_\_\_\_\_ Дата защиты \_\_\_\_\_

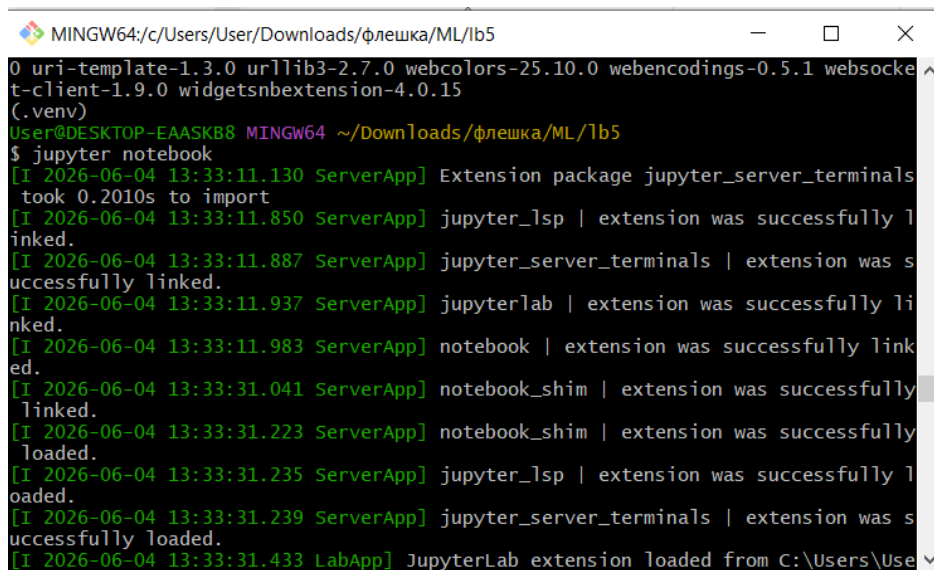
Ставрополь, 2026 г.

Тема: основные этапы исследовательского анализа данных.

Цель: научиться применять методы обработки данных в `pandas.DataFrame`, необходимые для разведочного анализа данных (EDA), включая работу с пропусками, выбросами, масштабирование и кодирование категориальных признаков.

Репозиторий: [https://github.com/TL-hash/ML\\_lab6](https://github.com/TL-hash/ML_lab6)

### Практическая часть:



```
MINGW64/c:/Users/User/Downloads/флешка/ML/lb5
0 uri-template-1.3.0 urllib3-2.7.0 webcolors-25.10.0 webencodings-0.5.1 websocke
t-client-1.9.0 widgetsnbextension-4.0.15
(.venv)
User@DESKTOP-EAASKB8 MINGW64 ~/Downloads/флешка/ML/lb5
$ jupyter notebook
[I 2026-06-04 13:33:11.130 ServerApp] Extension package jupyter_server_terminals
took 0.2010s to import
[I 2026-06-04 13:33:11.850 ServerApp] jupyter_lsp | extension was successfully l
inked.
[I 2026-06-04 13:33:11.887 ServerApp] jupyter_server_terminals | extension was s
uccessfully linked.
[I 2026-06-04 13:33:11.937 ServerApp] jupyterlab | extension was successfully li
nked.
[I 2026-06-04 13:33:11.983 ServerApp] notebook | extension was successfully link
ed.
[I 2026-06-04 13:33:31.041 ServerApp] notebook_shim | extension was successfully
linked.
[I 2026-06-04 13:33:31.223 ServerApp] notebook_shim | extension was successfully
loaded.
[I 2026-06-04 13:33:31.235 ServerApp] jupyter_lsp | extension was successfully l
oaded.
[I 2026-06-04 13:33:31.239 ServerApp] jupyter_server_terminals | extension was s
uccessfully loaded.
[I 2026-06-04 13:33:31.433 LabApp] JupyterLab extension loaded from C:\Users\Use
```

Рисунок 1 - Использование виртуального окружения для работы

### Примеры.

1. Создание `DataFrame` с пропущенными значениями. Создание небольшого `DataFrame` с данными о сотрудниках, содержащего пропуски (`NaN`) в столбцах "Возраст" и "Город". Демонстрирует, как `pandas` отображает пропущенные значения.

Листинг примера:

```
import pandas as pd
```

```
data = {
```

```
    "Имя": ["Анна", "Иван", "Ольга", None, "Мария"],
```

```
"Возраст": [25, 30, None, 40, 35],  
"Город": ["Москва", None, "Казань", "Новосибирск", "СПб"]  
}
```

```
df = pd.DataFrame(data)  
print(df)
```

Результат выполнения:

|   | Имя   | Возраст | Город       |
|---|-------|---------|-------------|
| 0 | Анна  | 25.0    | Москва      |
| 1 | Иван  | 30.0    | NaN         |
| 2 | Ольга | NaN     | Казань      |
| 3 | NaN   | 40.0    | Новосибирск |
| 4 | Мария | 35.0    | СПб         |

2. Обнаружение пропусков с `.isna()`. Метод `.isna()` возвращает булеву маску того же размера, что и `DataFrame`, где `True` указывает на пропущенное значение (`NaN`). Это позволяет визуально определить местоположение пропусков.

Листинг примера:

```
print(df.isna())
```

Результат выполнения:

|   | Имя   | Возраст | Город |
|---|-------|---------|-------|
| 0 | False | False   | False |
| 1 | False | False   | True  |
| 2 | False | True    | False |
| 3 | True  | False   | False |
| 4 | False | False   | False |

3. Подсчет пропусков с `.isna().sum()`. В сочетании с `.sum()` метод `.isna()` подсчитывает количество пропущенных значений в каждом столбце. Это позволяет быстро оценить, какие столбцы требуют обработки.

Листинг примера:

```
print(df.isna().sum())
```

Результат выполнения:

```
Имя      1
Возраст   1
Город     1
dtype: int64
```

4. Обнаружение непустых значений с `.notna()`. Метод `.notna()` является противоположностью `.isna()`: возвращает `True` для непустых значений. Полезен для фильтрации и анализа заполненности данных.

Листинг примера:

```
print(df.notna())
```

Результат выполнения:

```
Имя  Возраст  Город
0  True    True   True
1  True    True  False
2  True   False   True
3 False    True   True
4  True    True   True
```

5. Подсчет заполненных значений с `.notna().sum()`. Метод `.notna().sum()` подсчитывает количество непустых значений в каждом столбце. Это позволяет оценить полноту данных и процент заполнения.

Листинг примера:

```
print(df.notna().sum())
```

Результат выполнения:

```
Имя      4
Возраст   4
Город     4
dtype: int64
```

6. Создание расширенного DataFrame с пропусками. Создание более сложного DataFrame с данными о 10 сотрудниках, содержащего пропуски в различных столбцах: "Возраст", "Город", "Доход", "Образование", "Стаж работы". Демонстрирует реальную структуру данных с множественными пропусками.

Листинг примера:

```
import numpy as np

np.random.seed(42)

data = {
    "Имя": ["Анна", "Иван", "Ольга", "Петр", "Мария", "Дмитрий",
            "Елена", "Сергей", "Алина", "Артем"],
    "Возраст": [25, np.nan, 22, 40, 35, np.nan, 28, 31, np.nan, 29],
    "Город": ["Москва", "СПб", np.nan, "Новосибирск", "СПб",
              "Екатеринбург", np.nan, "Казань", "Томск", np.nan],
    "Доход": [50000, 60000, np.nan, 70000, 65000, 55000, 48000,
              np.nan, 52000, 58000],
    "Образование": [np.nan, "Высшее", "Среднее", "Высшее", np.nan,
                    "Высшее", "Среднее", "Высшее", np.nan, "Среднее"],
    "Стаж работы": [3, 8, 1, 15, 10, np.nan, 5, 7, np.nan, 2]
}
```

```
df = pd.DataFrame(data)
print(df)
```

Результат выполнения:

|   | Имя     | Возраст | Город        | Доход   | Образование | Стаж работы |
|---|---------|---------|--------------|---------|-------------|-------------|
| 0 | Анна    | 25.0    | Москва       | 50000.0 | NaN         | 3.0         |
| 1 | Иван    | NaN     | СПб          | 60000.0 | Высшее      | 8.0         |
| 2 | Ольга   | 22.0    | NaN          | NaN     | Среднее     | 1.0         |
| 3 | Петр    | 40.0    | Новосибирск  | 70000.0 | Высшее      | 15.0        |
| 4 | Мария   | 35.0    | СПб          | 65000.0 | NaN         | 10.0        |
| 5 | Дмитрий | NaN     | Екатеринбург | 55000.0 | Высшее      | NaN         |
| 6 | Елена   | 28.0    | NaN          | 48000.0 | Среднее     | 5.0         |
| 7 | Сергей  | 31.0    | Казань       | NaN     | Высшее      | 7.0         |
| 8 | Алина   | NaN     | Томск        | 52000.0 | NaN         | NaN         |
| 9 | Артем   | 29.0    | NaN          | 58000.0 | Среднее     | 2.0         |

7. Визуализация пропусков с помощью матрицы `missingno`. Библиотека `missingno` позволяет визуализировать структуру пропусков в данных. `msno.matrix()` отображает матрицу пропусков, где белые линии указывают на отсутствующие значения.

Листинг примера:

```
import missingno as msno
import matplotlib.pyplot as plt

msno.matrix(df)
plt.show()
```

Результат выполнения:

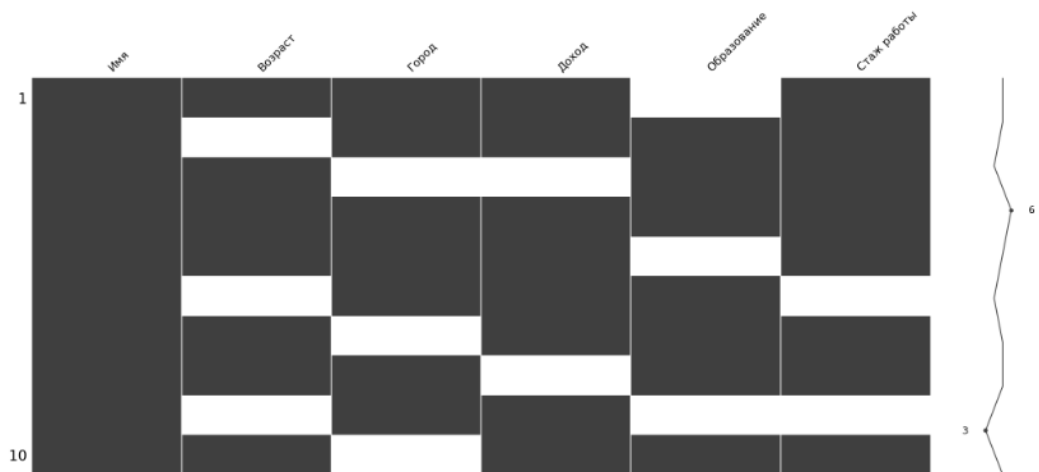


Рисунок 2 - Результат выполнения программы

8. Визуализация пропусков с помощью барчарта `missingno.misno.bar()` отображает столбчатую диаграмму, показывающую количество и процент пропусков в каждом столбце. Позволяет быстро оценить, какие столбцы содержат больше всего пропусков.

Листинг примера:

```
misno.bar(df)
plt.show()
```

Результат выполнения:

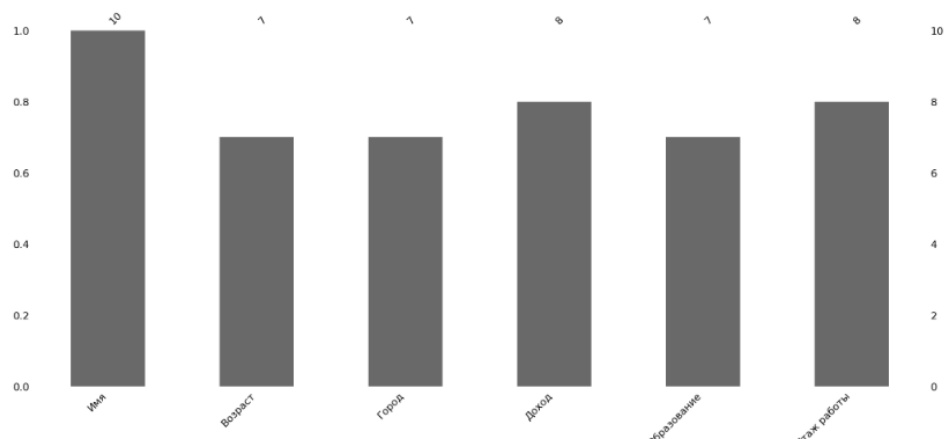


Рисунок 3 - Результат выполнения программы

9. Создание датасета Diamond (алмазы). Загрузка датасета Diamond из интернета (файл CSV). Датасет содержит информацию об алмазах: вес в

каратах, качество огранки, цвет, чистота, размеры и другие характеристики. Датасет используется для демонстрации полного цикла EDA.

Листинг примера:

```
df = pd.read_csv("https://raw.githubusercontent.com/Pratik-Bhujade/Diamond-Dataset/refs/heads/master/DiamondData.csv")
df.head()
```

Результат выполнения:

|   | carat | cut       | color | clarity | depth | table | price   | x    | y    | z    |
|---|-------|-----------|-------|---------|-------|-------|---------|------|------|------|
| 0 | 1.50  | Fair      | G     | SI1     | 64.5  | 57.0  | 10352.0 | 7.15 | 7.09 | 4.59 |
| 1 | 0.70  | Ideal     | E     | VS2     | 61.4  | 57.0  | 2274.0  | 5.72 | 5.78 | 3.53 |
| 2 | 1.22  | Premium   | G     | VS1     | 61.3  | 58.0  | 8779.0  | 6.91 | 6.89 | 4.23 |
| 3 | 0.51  | Premium   | E     | VS2     | 62.5  | 60.0  | 1590.0  | 5.08 | 5.10 | 3.18 |
| 4 | 2.02  | Very Good | J     | SI2     | 59.2  | 60.0  | 11757.0 | 8.27 | 8.39 | 4.91 |

Рисунок 4 - Результат выполнения программы

10. Удаление целевого признака price. Для демонстрации EDA (без построения модели) целевой признак price удаляется из датасета. Это позволяет сосредоточиться на анализе признаков без целевой переменной.

Листинг примера:

```
df.drop(columns=["price"], inplace=True)
df.head()
```

Результат выполнения:



|   | carat | cut       | color | clarity | depth | table | x    | y    | z    |
|---|-------|-----------|-------|---------|-------|-------|------|------|------|
| 0 | 1.50  | Fair      | G     | SI1     | 64.5  | 57.0  | 7.15 | 7.09 | 4.59 |
| 1 | 0.70  | Ideal     | E     | VS2     | 61.4  | 57.0  | 5.72 | 5.78 | 3.53 |
| 2 | 1.22  | Premium   | G     | VS1     | 61.3  | 58.0  | 6.91 | 6.89 | 4.23 |
| 3 | 0.51  | Premium   | E     | VS2     | 62.5  | 60.0  | 5.08 | 5.10 | 3.18 |
| 4 | 2.02  | Very Good | J     | SI2     | 59.2  | 60.0  | 8.27 | 8.39 | 4.91 |

Рисунок 5 - Результат выполнения программы

11. Информация о DataFrame (.info()). Метод .info() выводит сводную информацию о DataFrame: количество строк, названия столбцов, количество непустых значений, типы данных, использование памяти. Позволяет оценить структуру данных и выявить пропуски.

Листинг примера:

df.info()

Результат выполнения:

```
<class 'pandas.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   carat       50000 non-null  float64
1   cut         50000 non-null  str
2   color       50000 non-null  str
3   clarity     50000 non-null  str
4   depth       49529 non-null  float64
5   table       49610 non-null  float64
6   x           49779 non-null  float64
7   y           49667 non-null  float64
8   z           49572 non-null  float64
dtypes: float64(6), str(3)
memory usage: 3.4 MB
```

Рисунок 6 - Результат выполнения программы

12. Статистическое описание числовых признаков (.describe()). Метод .describe() вычисляет основные статистические характеристики числовых признаков: количество, среднее, стандартное отклонение,

минимум, квартили, максимум. Позволяет выявить аномальные значения и понять распределение данных.

Листинг примера:

```
df.describe().T
```

Результат выполнения:

|              | count   | mean      | std      | min  | 25%   | 50%   | 75%   | max       |
|--------------|---------|-----------|----------|------|-------|-------|-------|-----------|
| <b>carat</b> | 50000.0 | 0.906952  | 2.069722 | 0.2  | 0.40  | 0.70  | 1.05  | 49.990359 |
| <b>depth</b> | 49529.0 | 61.748402 | 1.429476 | 43.0 | 61.00 | 61.80 | 62.50 | 79.000000 |
| <b>table</b> | 49610.0 | 57.464787 | 2.242345 | 43.0 | 56.00 | 57.00 | 59.00 | 95.000000 |
| <b>x</b>     | 49779.0 | 5.731776  | 1.122125 | 0.0  | 4.71  | 5.70  | 6.54  | 10.230000 |
| <b>y</b>     | 49667.0 | 5.733675  | 1.119993 | 0.0  | 4.72  | 5.71  | 6.54  | 31.800000 |
| <b>z</b>     | 49572.0 | 3.539514  | 0.706533 | 0.0  | 2.91  | 3.53  | 4.04  | 31.800000 |

Рисунок 7 - Результат выполнения программы

13. Подсчет пропусков (`.isna().sum()`). Определение количества пропущенных значений в каждом столбце. Позволяет выявить, какие столбцы требуют обработки пропусков. В данном датасете пропуски есть в столбцах `depth`, `table`, `x`, `y`, `z`.

Листинг примера:

```
df.isna().sum()
```

Результат выполнения:

```
carat      0
cut         0
color       0
clarity     0
depth     471
table      390
x          221
y          333
z          428
dtype: int64
```

Рисунок 8 - Результат выполнения программы

14. Визуализация пропусков `missingno`. Визуализация структуры пропусков в датасете `Diamond` с помощью матрицы пропусков. Белые линии показывают расположение пропущенных значений.

Листинг примера:

```
msno.matrix(df)
```

Результат выполнения:

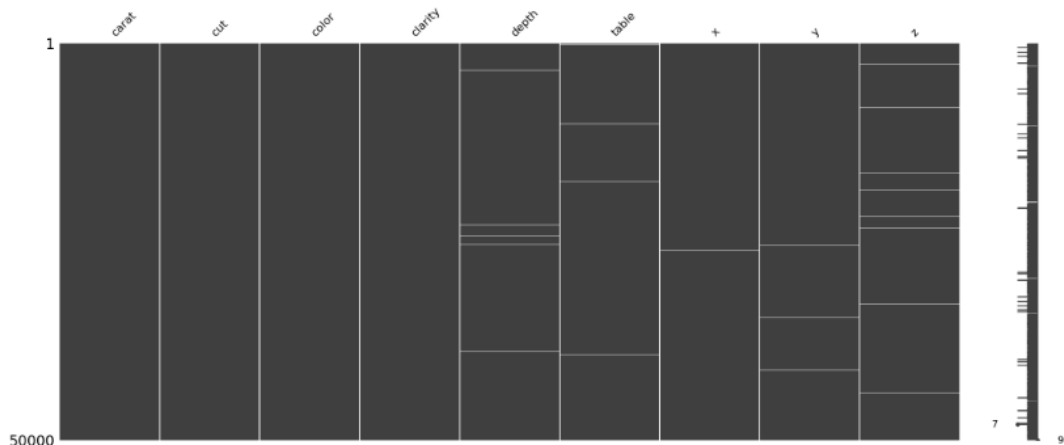


Рисунок 9 - Результат выполнения программы

15. Удаление строк с пропусками (`.dropna()`). Удаление всех строк, содержащих хотя бы одно пропущенное значение. После удаления проверяется, что пропусков больше нет.

Листинг примера:

```
df = df.dropna()
df.isna().sum()
```

Результат выполнения:

```
carat    0
cut      0
color    0
clarity  0
depth    0
```

```
table    0
x        0
y        0
z        0
dtype: int64
```

16. Визуализация выбросов с помощью boxplot. Построение ящиков с усами (boxplot) для всех числовых признаков. Boxplot позволяет визуально обнаружить выбросы – точки, находящиеся за пределами "усов" (за границами  $Q1 - 1.5 \times IQR$  и  $Q3 + 1.5 \times IQR$ ).

Листинг примера:

```
import seaborn as sns
```

```
numeric_cols = df.select_dtypes(include='number').columns
```

```
for col in numeric_cols:
```

```
    plt.figure(figsize=(10,4))
```

```
    sns.boxplot(x=df[col])
```

```
    plt.title(f"Boxplot: {col}")
```

```
    plt.show()
```

Результат выполнения:

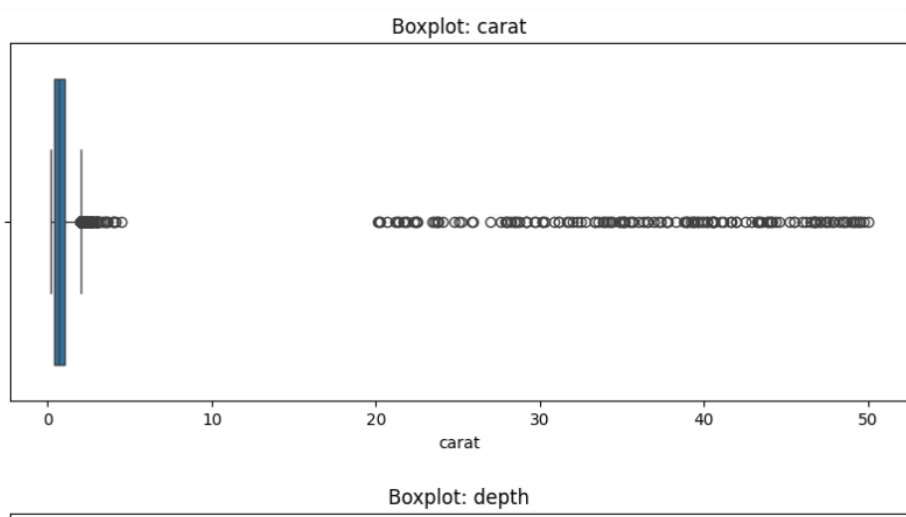


Рисунок 10 - Результат выполнения программы

17. Удаление выбросов методом IQR. Функция `remove_outliers_iqr` удаляет выбросы по заданному столбцу, используя межквартильный размах (IQR). Выбросами считаются значения, выходящие за пределы интервала  $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ . Применяется ко всем числовым столбцам.

Листинг примера:

```
def remove_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5*IQR
    upper = Q3 + 1.5*IQR
    return data[(data[column] >= lower) & (data[column] <= upper)]

for col in ["carat", "depth", "table", "x", "y", "z"]:
    df = remove_outliers_iqr(df, col)

df.head()
```

Результат выполнения:

|   | carat | cut       | color | clarity | depth | table | x    | y    | z    |
|---|-------|-----------|-------|---------|-------|-------|------|------|------|
| 0 | 1.50  | Fair      | G     | SI1     | 64.5  | 57.0  | 7.15 | 7.09 | 4.59 |
| 1 | 0.70  | Ideal     | E     | VS2     | 61.4  | 57.0  | 5.72 | 5.78 | 3.53 |
| 2 | 1.22  | Premium   | G     | VS1     | 61.3  | 58.0  | 6.91 | 6.89 | 4.23 |
| 3 | 0.51  | Premium   | E     | VS2     | 62.5  | 60.0  | 5.08 | 5.10 | 3.18 |
| 4 | 2.02  | Very Good | J     | SI2     | 59.2  | 60.0  | 8.27 | 8.39 | 4.91 |

Рисунок 11 - Результат выполнения программы

18. Стандартизация числовых признаков (StandardScaler). Стандартизация преобразует данные так, чтобы они имели среднее 0 и стандартное отклонение 1 по формуле  $z = (x - \mu) / \sigma$ . Это необходимо для алгоритмов машинного обучения, чувствительных к масштабу признаков (SVM, KNN, линейная регрессия).

Листинг примера:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])
df.head()
```

Результат выполнения:

|   | carat     | cut       | color | clarity | depth     | table     | x         | y         | z         |
|---|-----------|-----------|-------|---------|-----------|-----------|-----------|-----------|-----------|
| 0 | 1.776201  | Fair      | G     | SI1     | 2.447919  | -0.141333 | 1.432148  | 1.378806  | 1.695091  |
| 1 | -0.127819 | Ideal     | E     | VS2     | -0.345564 | -0.141333 | 0.074730  | 0.128209  | 0.066886  |
| 2 | 1.109794  | Premium   | G     | VS1     | -0.435676 | 0.352009  | 1.204330  | 1.187875  | 1.142116  |
| 3 | -0.580024 | Premium   | E     | VS2     | 0.645672  | 1.338693  | -0.532787 | -0.520956 | -0.470728 |
| 4 | 3.013814  | Very Good | J     | SI2     | -2.328036 | 1.338693  | 2.495301  | 2.619857  | 2.186624  |

Рисунок 12 - Результат выполнения программы

19. Label Encoding для признака cut (качество огранки). Преобразование порядкового категориального признака cut в числовые метки. Создается словарь соответствия: Fair-0, Good-1, Very Good-2, Premium-3, Ideal-4. Это позволяет сохранить порядок категорий.

Листинг примера:

```
cut_mapping = {  
    "Fair": 0, "Good": 1, "Very Good": 2, "Premium": 3, "Ideal": 4  
}  
df["cut"] = df["cut"].map(cut_mapping)
```

20. Label Encoding для признака clarity (чистота). Преобразование порядкового категориального признака clarity в числовые метки. Словарь отображает категории чистоты (от I1 до IF) в числа от 0 до 7. Чем выше число, тем выше качество чистоты.

Листинг примера:

```
clarity_mapping = {  
    "I1": 0, "SI2": 1, "SI1": 2, "VS2": 3,  
    "VS1": 4, "VVS2": 5, "VVS1": 6, "IF": 7  
}  
df["clarity"] = df["clarity"].map(clarity_mapping)  
df.head()
```

Результат выполнения:

|   | carat     | cut | color | clarity | depth     | table     | x         | y         | z         |
|---|-----------|-----|-------|---------|-----------|-----------|-----------|-----------|-----------|
| 0 | 1.776201  | 0.0 | G     | 2       | 2.447919  | -0.141333 | 1.432148  | 1.378806  | 1.695091  |
| 1 | -0.127819 | 4.0 | E     | 3       | -0.345564 | -0.141333 | 0.074730  | 0.128209  | 0.066886  |
| 2 | 1.109794  | 3.0 | G     | 4       | -0.435676 | 0.352009  | 1.204330  | 1.187875  | 1.142116  |
| 3 | -0.580024 | 3.0 | E     | 3       | 0.645672  | 1.338693  | -0.532787 | -0.520956 | -0.470728 |
| 4 | 3.013814  | 2.0 | J     | 1       | -2.328036 | 1.338693  | 2.495301  | 2.619857  | 2.186624  |

Рисунок 13 - Результат выполнения программы

21. One-Hot Encoding для признака color (цвет). Применение one-hot-кодирования к номинативному признаку color. Параметр drop\_first=True удаляет первый бинарный столбец для предотвращения дамми-ловушки. В результате создаются бинарные столбцы для каждого цвета, кроме первого (D).

Листинг примера:

```
df = pd.get_dummies(df, columns=["color"], drop_first=True)
print(df.head())
```

Результат выполнения:

|   | carat     | cut | clarity | depth     | table     | x         | y         | z         | \ |
|---|-----------|-----|---------|-----------|-----------|-----------|-----------|-----------|---|
| 0 | 1.776201  | 0.0 | 2       | 2.447919  | -0.141333 | 1.432148  | 1.378806  | 1.695091  |   |
| 1 | -0.127819 | 4.0 | 3       | -0.345564 | -0.141333 | 0.074730  | 0.128209  | 0.066886  |   |
| 2 | 1.109794  | 3.0 | 4       | -0.435676 | 0.352009  | 1.204330  | 1.187875  | 1.142116  |   |
| 3 | -0.580024 | 3.0 | 3       | 0.645672  | 1.338693  | -0.532787 | -0.520956 | -0.470728 |   |
| 4 | 3.013814  | 2.0 | 1       | -2.328036 | 1.338693  | 2.495301  | 2.619857  | 2.186624  |   |

|   | color_E | color_F | color_G | color_H | color_I | color_J |
|---|---------|---------|---------|---------|---------|---------|
| 0 | False   | False   | True    | False   | False   | False   |
| 1 | True    | False   | False   | False   | False   | False   |
| 2 | False   | False   | True    | False   | False   | False   |
| 3 | True    | False   | False   | False   | False   | False   |
| 4 | False   | False   | False   | False   | False   | True    |

Рисунок 14 - Результат выполнения программы

## Задания.

Задание 1. Обнаружение и обработка пропущенных значений.

Датасет: titanic (пассажиры Титаника).

Источник: seaborn.load\_dataset("titanic").

Инструкции:

1. Загрузите датасет titanic.
2. Определите количество пропущенных значений в каждом столбце.
3. Визуализируйте пропуски с помощью библиотеки missigno.



4. Заполните пропущенные значения: признак `age` – средним значением, признак `embarked` – наиболее частным значением, признак `deck` – удалите.

5. Отобразите информацию о таблице до и после обработки (`.info()`, `.isna()`, `.sum()`).

Решение:

1. Загрузка данных. `pd.read_csv()` загружает датасет `Titanic` из интернета. Датасет содержит информацию о пассажирах: возраст, пол, класс билета, порт посадки и другие характеристики.

2. Определение пропусков. `df.isna().sum()` подсчитывает количество пропущенных значений (`NaN`) в каждом столбце. Это позволяет увидеть, какие столбцы требуют обработки.

3. Визуализация пропусков. `msno.matrix()` показывает матрицу пропусков (белые линии = пропуски). `msno.bar()` отображает столбчатую диаграмму количества пропусков в каждом столбце.

4. Заполнение пропусков: `df['Age'].fillna(df['Age'].mean())` – заменяет пропуски в возрасте на среднее арифметическое значение (29.7 лет), `df['Embarked'].fillna(df['Embarked'].mode()[0])` – заменяет пропуски в порте посадки на самое частое значение ('S'), `df.drop(columns=['Cabin'])` – удаляет столбец `Cabin` (информация о каюте), так как в нем слишком много пропусков (687 из 891).

5. Отображение информации. `.info()` выводит сводку по `DataFrame` (количество строк, столбцов, типы данных, количество непустых значений). `.isna().sum()` показывает количество пропусков в каждом столбце до и после обработки.

Листинг задания:

```
import pandas as pd
import numpy as np
import missingno as msno
import matplotlib.pyplot as plt
```

```

# 1. Загрузите датасет titanic
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

# 2. Определите количество пропущенных значений в каждом столбце
print("\n ДО ОБРАБОТКИ ")
print(f"Размер датасета: {df.shape[0]} строк, {df.shape[1]} столбцов")
print("\n Количество пропущенных значений в каждом столбце:")
print(df.isna().sum())

# 3. Визуализируйте пропуски с помощью библиотеки missingno
print("\n ВИЗУАЛИЗАЦИЯ ПРОПУСКОВ ")

# Матрица пропусков
plt.figure(figsize=(10, 4))
msno.matrix(df, figsize=(10, 4))
plt.title('Матрица пропущенных значений (белые линии = пропуски)')
plt.show()

# 4. Заполните пропущенные значения
print("\n ОБРАБОТКА ПРОПУСКОВ ")

# Признак age - средним значением
age_mean = df['Age'].mean()
print(f"Средний возраст пассажиров: {age_mean:.1f} лет")
df['Age'] = df['Age'].fillna(age_mean)

# Признак embarked - наиболее частым значением
embarked_mode = df['Embarked'].mode()[0]
print(f"Наиболее частый порт посадки: '{embarked_mode}'")
df['Embarked'] = df['Embarked'].fillna(embarked_mode)

# Признак deck (Cabin) - удалите
print("Удаляем столбец 'Cabin' (информация о каюте)")
df = df.drop(columns=['Cabin'])

# 5. Отобразите информацию о таблице до и после обработки
print("\n ИНФОРМАЦИЯ О ТАБЛИЦЕ ДО ОБРАБОТКИ ")
print("Метод .info():")
print(df.info())

print("\n Метод .isna().sum() ДО обработки:")
df_original = pd.read_csv(url)
print(df_original.isna().sum())

print("\n ИНФОРМАЦИЯ О ТАБЛИЦЕ ПОСЛЕ ОБРАБОТКИ ")
print("Метод .info():")
print(df.info())

print("\nМетод .isna().sum() ПОСЛЕ обработки:")
print(df.isna().sum())

```

```
# Итоговая проверка
print("\n ИТОГ ")
if df.isna().sum().sum() == 0:
    print("Все пропуски успешно обработаны!")
else:
    print(f"Осталось пропусков: {df.isna().sum().sum()}")
```

## Результат выполнения:

```
ДО ОБРАБОТКИ
Размер датасета: 891 строк, 12 столбцов

Количество пропущенных значений в каждом столбце:
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age             177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin           687
Embarked         2
dtype: int64
```

### ВИЗУАЛИЗАЦИЯ ПРОПУСКОВ

<Figure size 1000x400 with 0 Axes>

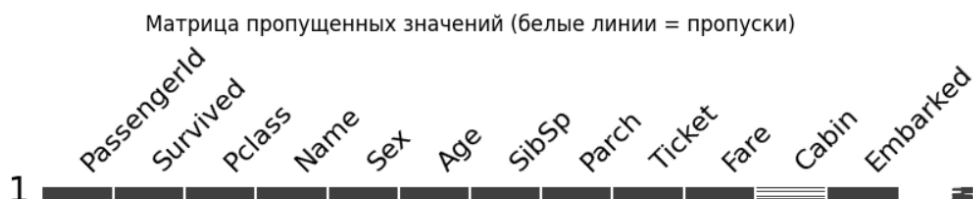


Рисунок 15 - Результат выполнения программы

## Задание 2. Обнаружение и удаление выбросов.

Датасет: penguins (описание антарктических пингвинов).

Источник: seaborn.load\_dataset("penguins").

### Инструкции:

1. Загрузите датасет penguins.
2. Постройте boxplot-графики для признаков bill\_length\_mm, bill\_depth\_mm, flipper\_length\_mm, body\_mass\_g.
3. Используя метод межквартильного размаха (IQR), выявите и удалите выбросы по каждому из указанных признаков.
4. Сравните размеры датасета до и после фильтрации.
5. Постройте boxplot-график до и после удаления выбросов для одного из признаков.

Решение:

1. Загрузка данных. `sns.load_dataset("penguins")` загружает датасет с данными о пингвинах: длина клюва, глубина клюва, длина крыла, масса тела и другие характеристики.
2. Построение `boxplot`-графиков. `Boxplot` (ящик с усами) показывает распределение данных: медиану, квартили и выбросы. Точки за пределами "усов" считаются выбросами.
3. Метод IQR. Межквартильный размах (IQR) – это разница между третьим (Q3) и первым (Q1) квартилями. Выбросами считаются значения, выходящие за пределы: нижняя граница:  $Q1 - 1.5 \times IQR$ , верхняя граница:  $Q3 + 1.5 \times IQR$ .
4. Удаление выбросов. Для каждого признака определяются границы, и из датасета удаляются строки, содержащие выбросы хотя бы по одному признаку.
5. Сравнение размеров. Выводится количество строк до и после удаления выбросов, а также процент удаленных данных.
6. Визуализация. Строятся `boxplot`-графики для одного из признаков (например, `bill_length_mm`) до и после удаления выбросов для наглядного сравнения.

Листинг задания:

```
import seaborn as sns
```

```
# 1. Загрузка датасета penguins  
df = sns.load_dataset("penguins")
```

```
print(" ДАТАСЕТ PENGUINS ")  
print(f"Размер датасета: {df.shape[0]} строк, {df.shape[1]} столбцов")  
print("\n Первые 5 строк:")  
print(df.head())
```

```
# 2. Построение boxplot-графиков для указанных признаков  
print("\n BOXPLOT-ГРАФИКИ ДО УДАЛЕНИЯ ВЫБРОСОВ ")
```

```
features = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
```

```
plt.figure(figsize=(12, 8))  
for i, feature in enumerate(features, 1):
```

```
plt.subplot(2, 2, i)
sns.boxplot(y=df[feature], color='skyblue')
plt.title(f'Boxplot признака "{feature}"')
plt.ylabel(feature)
plt.tight_layout()
plt.show()
```

```
# 3. Удаление выбросов методом IQR
print("\n УДАЛЕНИЕ ВЫБРОСОВ МЕТОДОМ IQR ")
```

```
# Сохраняем копию исходного датасета
df_original = df.copy()
```

```
# Функция для поиска выбросов по IQR
def find_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
```

```
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
```

```
    outliers = data[(data[column] < lower_bound) | (data[column] > upper_bound)]
```

```
    print(f"\n Признак '{column}':")
    print(f"  Q1 = {Q1:.2f}, Q3 = {Q3:.2f}, IQR = {IQR:.2f}")
    print(f"  Нижняя граница: {lower_bound:.2f}")
    print(f"  Верхняя граница: {upper_bound:.2f}")
    print(f"  Количество выбросов: {len(outliers)}")
```

```
    return lower_bound, upper_bound
```

```
# Удаляем выбросы из датасета
df_clean = df.copy()
```

```
for feature in features:
    lower, upper = find_outliers_iqr(df_clean, feature)
    df_clean = df_clean[(df_clean[feature] >= lower) & (df_clean[feature] <= upper)]
```

```
# 4. Сравнение размеров датасета до и после фильтрации
print("\n СРАВНЕНИЕ РАЗМЕРОВ ДАТАСЕТА ")
print(f"Размер до удаления выбросов: {len(df_original)} строк")
print(f"Размер после удаления выбросов: {len(df_clean)} строк")
print(f"Удалено строк: {len(df_original) - len(df_clean)}")
print(f"Удалено процентов: {(len(df_original) - len(df_clean)) / len(df_original) * 100:.2f}%")
```

```
# 5. Построение boxplot-графика до и после удаления выбросов для одного признака
print("\n СРАВНЕНИЕ BOXPLOT-ГРАФИКОВ ДО И ПОСЛЕ ")
```

```
selected_feature = 'bill_length_mm'
```

```

plt.figure(figsize=(12, 5))

# График до удаления выбросов
plt.subplot(1, 2, 1)
sns.boxplot(y=df_original[selected_feature], color='skyblue')
plt.title(f'ДО удаления выбросов\nПризнак "{selected_feature}"')
plt.ylabel(selected_feature)

# График после удаления выбросов
plt.subplot(1, 2, 2)
sns.boxplot(y=df_clean[selected_feature], color='lightgreen')
plt.title(f'ПОСЛЕ удаления выбросов\nПризнак "{selected_feature}"')
plt.ylabel(selected_feature)

plt.tight_layout()
plt.show()

print("\n=== ИТОГ ===")
print(f'Выбросы успешно удалены методом IQR!')
print(f'Исходный датасет: {len(df_original)} строк')
print(f'Очищенный датасет: {len(df_clean)} строк')

```

## Результат выполнения:

ДАТАСЕТ PENGUINS  
Размер датасета: 344 строк, 7 столбцов

Первые 5 строк:

|   | species | island    | bill_length_mm | bill_depth_mm | flipper_length_mm | \ |
|---|---------|-----------|----------------|---------------|-------------------|---|
| 0 | Adelie  | Torgersen | 39.1           | 18.7          | 181.0             |   |
| 1 | Adelie  | Torgersen | 39.5           | 17.4          | 186.0             |   |
| 2 | Adelie  | Torgersen | 40.3           | 18.0          | 195.0             |   |
| 3 | Adelie  | Torgersen | NaN            | NaN           | NaN               |   |
| 4 | Adelie  | Torgersen | 36.7           | 19.3          | 193.0             |   |

|   | body_mass_g | sex    |
|---|-------------|--------|
| 0 | 3750.0      | Male   |
| 1 | 3800.0      | Female |
| 2 | 3250.0      | Female |
| 3 | NaN         | NaN    |
| 4 | 3450.0      | Female |

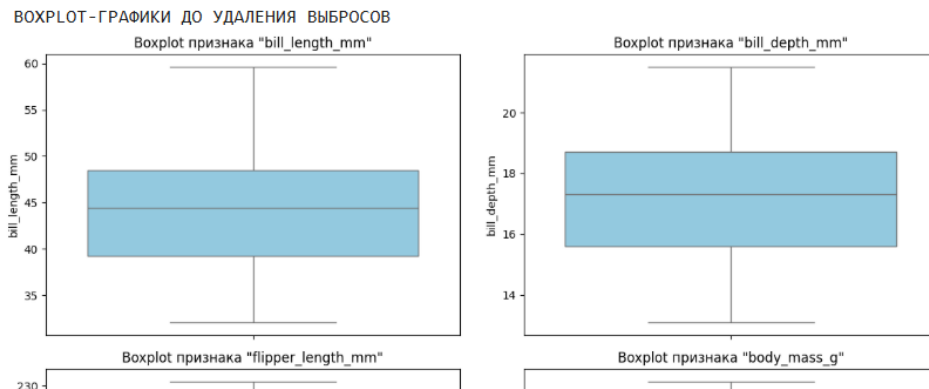


Рисунок 16 - Результат выполнения программы

## Задание 3. Масштабирование числовых признаков.

Датасет: California housing.

Источник: `from sklearn.datasets import fetch_california_housing`.

Инструкции:

1. Загрузите данные с помощью `fetch_california_housing` (`as_frame=True`).
2. Преобразуйте данные в `pandas.DataFrame`.
3. Выполните: стандартизацию признаков с помощью `StandardScaler`, нормализацию в диапазон `[0, 1]` с помощью `MinMaxScaler` (на копии таблицы).
4. Постройте гистограммы распределения признака `MedInc` до и после масштабирования.
5. Сравните поведение шкал на гистограммах.

Решение:

1. Загрузка данных. `fetch_california_housing(as_frame=True)` загружает датасет с информацией о жилье в Калифорнии. Параметр `as_frame=True` возвращает данные в виде `pandas.DataFrame`.
2. Преобразование в `DataFrame`. Данные уже загружаются в виде `DataFrame`, поэтому дополнительное преобразование не требуется.
3. Масштабирование признаков: стандартизация (`StandardScaler`) – преобразует данные так, чтобы они имели среднее значение 0 и стандартное отклонение 1:  $z = (x - \mu) / \sigma$ . Нормализация (`MinMaxScaler`) – преобразует данные в диапазон `[0, 1]`:  $x' = (x - \min) / (\max - \min)$ .
4. Визуализация. Гистограммы показывают распределение признака `MedInc` (медианный доход в районе) до и после масштабирования.
5. Сравнение шкал. Стандартизация изменяет значения, но сохраняет форму распределения (центрирует относительно 0). Нормализация сжимает данные в отрезок `[0, 1]`.

Листинг задания:

```
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler, MinMaxScaler
```

```

# 1. Загрузка данных
print(" ЗАГРУЗКА ДАННЫХ ")
data = fetch_california_housing(as_frame=True)

# 2. Преобразование в pandas.DataFrame
df = data.frame
print(f"Размер датасета: {df.shape[0]} строк, {df.shape[1]} столбцов")
print("\n Первые 5 строк:")
print(df.head())

# Описание признаков
print("\nОписание признаков:")
print(data.DESCR[:500])

# 3. Масштабирование признаков
print("\n МАСШТАБИРОВАНИЕ ПРИЗНАКОВ ")

df_standard = df.copy()
df_normalized = df.copy()

features = df.columns.tolist()
print(f"Признаки для масштабирования: {features}")

# Стандартизация (StandardScaler)
scaler_std = StandardScaler()
df_standard[features] = scaler_std.fit_transform(df[features])
print("\n Стандартизация выполнена (среднее = 0, std = 1)")

# Нормализация в диапазон [0, 1] (MinMaxScaler)
scaler_mm = MinMaxScaler()
df_normalized[features] = scaler_mm.fit_transform(df[features])
print(" Нормализация выполнена (диапазон [0, 1])")

# 4. Построение гистограмм для признака MedInc
print("\n ВИЗУАЛИЗАЦИЯ РАСПРЕДЕЛЕНИЯ ПРИЗНАКА MedInc ")

selected_feature = 'MedInc'

plt.figure(figsize=(15, 10))

# Гистограмма исходных данных
plt.subplot(3, 1, 1)
plt.hist(df[selected_feature], bins=50, color='skyblue', edgecolor='black', alpha=0.7)
plt.title(f'Исходные данные\nПризнак "{selected_feature}"')
plt.xlabel('Значение')
plt.ylabel('Частота')
plt.grid(True)

# Гистограмма после стандартизации
plt.subplot(3, 1, 2)

```



```

plt.hist(df_standard[selected_feature], bins=50, color='lightgreen', edgecolor='black',
alpha=0.7)
plt.title(f'После стандартизации (StandardScaler)\nsреднее =
{df_standard[selected_feature].mean():.2f}, '
f'std = {df_standard[selected_feature].std():.2f}')
plt.xlabel('Значение')
plt.ylabel('Частота')
plt.grid(True)

# Гистограмма после нормализации
plt.subplot(3, 1, 3)
plt.hist(df_normalized[selected_feature], bins=50, color='salmon', edgecolor='black',
alpha=0.7)
plt.title(f'После нормализации (MinMaxScaler)\nmin =
{df_normalized[selected_feature].min():.2f}, '
f'max = {df_normalized[selected_feature].max():.2f}')
plt.xlabel('Значение')
plt.ylabel('Частота')
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# 5. Сравнение поведения шкал
print("\n СРАВНЕНИЕ ШКАЛ ")
print("\nСтатистика для признака 'MedInc':")
print(f'{'Показатель':<20} {'Исходные':<15} {'Стандартизация':<15}
{'Нормализация':<15}")
print(f'{'Минимум':<20} {df[selected_feature].min():<15.4f} "
f"{df_standard[selected_feature].min():<15.4f} "
f"{df_normalized[selected_feature].min():<15.4f}")
print(f'{'Максимум':<20} {df[selected_feature].max():<15.4f} "
f"{df_standard[selected_feature].max():<15.4f} "
f"{df_normalized[selected_feature].max():<15.4f}")
print(f'{'Среднее':<20} {df[selected_feature].mean():<15.4f} "
f"{df_standard[selected_feature].mean():<15.4f} "
f"{df_normalized[selected_feature].mean():<15.4f}")
print(f'{'Станд. отклонение':<20} {df[selected_feature].std():<15.4f} "
f"{df_standard[selected_feature].std():<15.4f} "
f"{df_normalized[selected_feature].std():<15.4f}")

# 5. Анализ результатов
print("\n АНАЛИЗ РЕЗУЛЬТАТОВ ")
print("1. Исходные данные:")
print(" - Значения имеют произвольный масштаб и диапазон")
print(" - Распределение имеет выраженную правостороннюю асимметрию")

print("\n 2. Стандартизация (StandardScaler):")
print(" - Данные центрированы относительно 0 (среднее = 0)")
print(" - Единичное стандартное отклонение (std = 1)")
print(" - Форма распределения сохраняется (не меняет форму гистограммы)")
print(" - Диапазон значений не ограничен")

```

```

print("\n 3. Нормализация (MinMaxScaler):")
print(" - Данные сжаты в диапазон [0, 1]")
print(" - Сохраняется форма распределения")
print(" - Все значения находятся между 0 и 1")
print(" - Чувствительна к выбросам (максимальное значение становится 1)")

```

### Результат выполнения:

```

ЗАГРУЗКА ДАННЫХ
Размер датасета: 20640 строк, 9 столбцов

Первые 5 строк:
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0   8.3252    41.0    6.984127   1.023810     322.0    2.555556    37.88
1   8.3014    21.0    6.238137   0.971880    2401.0    2.109842    37.86
2   7.2574    52.0    8.288136   1.073446     496.0    2.802260    37.85
3   5.6431    52.0    5.817352   1.073059     558.0    2.547945    37.85
4   3.8462    52.0    6.281853   1.081081     565.0    2.181467    37.85

   Longitude  MedHouseVal
0   -122.23      4.526
1   -122.22      3.585
2   -122.24      3.521
3   -122.25      3.413
4   -122.25      3.422

Описание признаков:
.. _california_housing_dataset:

California Housing dataset
-----

**Data Set Characteristics:**

:Number of Instances: 20640

:Number of Attributes: 8 numeric, predictive attributes and the target

:Attribute Information:
  - MedInc      median income in block group
  - HouseAge    median house age in block group

```

Рисунок 17 - Результат выполнения программы

### Задание 4. Кодирование категориальных признаков.

Датасет: adult (перепись населения США, income dataset).

Источник: <https://archive.ics.uci.edu/ml/datasets/adult>. Или через библиотеку `sklearn.datasets.fetch_openml("adult")`.

#### Инструкции:

1. Загрузите данные и отберите признаки: категориальные – education, marital-status, occupation, целевой признак – income.
2. Проведите Label Encoding для признака education, предполагая, что уровни образования упорядочены.

3. Примените One-Hot Encoding к признакам marital-status и occupation.
4. Проверьте итоговую размерность таблицы до и после кодирования.
5. Убедитесь, что в one-hot-кодировании не присутствует дамми-ловушка.

Решение:

1. Загрузка данных. `fetch_openml("adult")` загружает датасет Adult, содержащий данные переписи населения США. Целевой признак – income (доход больше или меньше \$50К в год).
2. Отбор признаков. Выбираем категориальные признаки education, marital-status, occupation и целевой признак income.
3. Label Encoding – преобразует категориальные значения в числовые метки (0, 1, 2, ...). Применяется для порядковых признаков, где важен порядок категорий (например, начальное, среднее, высшее образование).
4. One-Hot Encoding – создает бинарные столбцы для каждой категории (1 – категория присутствует, 0 – отсутствует). Применяется для номинативных признаков, где порядок категорий не важен.
5. Проверка размерности. Сравниваем количество столбцов до и после кодирования.
6. Дамми-ловушка (dummy trap) – возникает, когда при one-hot-кодировании остаются все категорий, что приводит к мультиколлинеарности. Решается удалением одного из бинарных столбцов (`drop_first=True`).

Листинг задания:

```
from sklearn.datasets import fetch_openml
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# 1. Загрузка данных
print(" ЗАГРУЗКА ДАННЫХ ")
adult = fetch_openml("adult", version=2, as_frame=True)
df = adult.frame
```

```

print(f"Размер датасета: {df.shape[0]} строк, {df.shape[1]} столбцов")
print("\n Первые 5 строк:")
print(df.head())

# 2. Отбор признаков
print("\n ОТБОР ПРИЗНАКОВ ")
selected_features = ['education', 'marital-status', 'occupation']
target = 'class' # income (class)

df_selected = df[selected_features + [target]].copy()

print(f"Выбранные категориальные признаки: {selected_features}")
print(f"Целевой признак: {target}")
print(f"Размер таблицы: {df_selected.shape}")

print("\n Уникальные значения в признаке 'education':")
print(df_selected['education'].unique()[:10])
print(f"Всего уровней образования: {df_selected['education'].nunique()}")

print("\n Уникальные значения в признаке 'marital-status':")
print(df_selected['marital-status'].unique())

print("\n Уникальные значения в признаке 'occupation':")
print(df_selected['occupation'].unique()[:10])
print(f"Всего профессий: {df_selected['occupation'].nunique()}")

# 3. Label Encoding для признака education
print("\n LABEL ENCODING (education) ")
education_order = [
    'Preschool', '1st-4th', '5th-6th', '7th-8th', '9th',
    '10th', '11th', '12th', 'HS-grad', 'Some-college',
    'Assoc-voc', 'Assoc-acdm', 'Bachelors', 'Masters',
    'Prof-school', 'Doctorate'
]

education_mapping = {level: i for i, level in enumerate(education_order)}
df_selected['education_encoded'] = df_selected['education'].map(education_mapping)

print(f"Пример кодирования:")
for level in education_order[:5]:
    print(f" '{level}' -> {education_mapping[level]}")
print(f" ...")

print(f"\n Первые 10 значений после кодирования:")
print(df_selected[['education', 'education_encoded']].head(10))

# 4. One-Hot Encoding для marital-status и occupation
print("\n ONE-HOT ENCODING (marital-status, occupation) ")

dummies_marital = pd.get_dummies(
    df_selected['marital-status'],

```

```

        prefix='marital',
        drop_first=True
    )

    dummies_occupation = pd.get_dummies(
        df_selected['occupation'],
        prefix='occ',
        drop_first=True
    )

    print(f"Создано бинарных столбцов для 'marital-status': {dummies_marital.shape[1]}")
    print(f"Создано бинарных столбцов для 'occupation': {dummies_occupation.shape[1]}")

    df_encoded = pd.concat([
        df_selected['education_encoded'],
        dummies_marital,
        dummies_occupation,
        df_selected[target]
    ], axis=1)

    print("\n Первые 5 строк после кодирования:")
    print(df_encoded.head())

    # 4. Проверка итоговой размерности
    print("\n ПРОВЕРКА РАЗМЕРНОСТИ ")

    print(f"Размер таблицы ДО кодирования: {df_selected.shape[0]} строк x {df_selected.shape[1]} столбцов")
    print(f" - Категориальные признаки: {len(selected_features)}")
    print(f" - Целевой признак: 1")

    print(f"\n Размер таблицы ПОСЛЕ кодирования: {df_encoded.shape[0]} строк x {df_encoded.shape[1]} столбцов")
    print(f" - Label Encoding (education): 1 столбец")
    print(f" - One-Hot Encoding (marital-status): {dummies_marital.shape[1]} столбцов")
    print(f" - One-Hot Encoding (occupation): {dummies_occupation.shape[1]} столбцов")
    print(f" - Целевой признак: 1 столбец")

    # 5. Проверка на наличие дамми-ловушки
    print("\n ПРОВЕРКА ДАММИ-ЛОВУШКИ ")

    marital_cols = [col for col in df_encoded.columns if col.startswith('marital_')]
    if len(marital_cols) > 0:
        zero_rows = (df_encoded[marital_cols].sum(axis=1) == 0).sum()
        print(f"Для 'marital-status':")
        print(f" - Создано столбцов: {len(marital_cols)}")
        print(f" - Категорий в оригинале: {df_selected['marital-status'].nunique()}")
        print(f" - Строк, где все столбцы = 0: {zero_rows}")
        if zero_rows > 0:
            print(f" Дамми-ловушки нет (один столбец удален)")
        else:

```

```

print(f" Возможна дамми-ловушка (все столбцы сохранены)")

occ_cols = [col for col in df_encoded.columns if col.startswith('occ_')]
if len(occ_cols) > 0:
    zero_rows = (df_encoded[occ_cols].sum(axis=1) == 0).sum()
    print(f"\nДля 'occupation':")
    print(f" - Создано столбцов: {len(occ_cols)}")
    print(f" - Категорий в оригинале: {df_selected['occupation'].nunique()}")
    print(f" - Строк, где все столбцы = 0: {zero_rows}")
    if zero_rows > 0:
        print(f" Дамми-ловушки нет (один столбец удален)")
    else:
        print(f" Возможна дамми-ловушка (все столбцы сохранены)")

print("\n ИТОГ ")
print(f" Кодирование категориальных признаков выполнено!")
print(f" Исходная таблица: {df_selected.shape[0]} x {df_selected.shape[1]}")
print(f" Закодированная таблица: {df_encoded.shape[0]} x {df_encoded.shape[1]}")
print(f" Дамми-ловушка предотвращена с помощью drop_first=True")

```

## Результат выполнения:

### ЗАГРУЗКА ДАННЫХ

Размер датасета: 48842 строк, 15 столбцов

Первые 5 строк:

|   | age | workclass | fnlwgt | education    | education-num | marital-status     | \ |
|---|-----|-----------|--------|--------------|---------------|--------------------|---|
| 0 | 25  | Private   | 226802 | 11th         | 7             | Never-married      |   |
| 1 | 38  | Private   | 89814  | HS-grad      | 9             | Married-civ-spouse |   |
| 2 | 28  | Local-gov | 336951 | Assoc-acdm   | 12            | Married-civ-spouse |   |
| 3 | 44  | Private   | 160323 | Some-college | 10            | Married-civ-spouse |   |
| 4 | 18  | NaN       | 103497 | Some-college | 10            | Never-married      |   |

|   | occupation        | relationship | race  | sex    | capital-gain | capital-loss | \ |
|---|-------------------|--------------|-------|--------|--------------|--------------|---|
| 0 | Machine-op-inspct | Own-child    | Black | Male   | 0            | 0            |   |
| 1 | Farming-fishing   | Husband      | White | Male   | 0            | 0            |   |
| 2 | Protective-serv   | Husband      | White | Male   | 0            | 0            |   |
| 3 | Machine-op-inspct | Husband      | Black | Male   | 7688         | 0            |   |
| 4 | NaN               | Own-child    | White | Female | 0            | 0            |   |

|   | hours-per-week | native-country | class |
|---|----------------|----------------|-------|
| 0 | 40             | United-States  | <=50K |
| 1 | 50             | United-States  | <=50K |
| 2 | 40             | United-States  | >50K  |
| 3 | 40             | United-States  | >50K  |
| 4 | 30             | United-States  | <=50K |

### ОТБОР ПРИЗНАКОВ

Выбранные категориальные признаки: ['education', 'marital-status', 'occupation']

Целевой признак: class

Размер таблицы: (48842, 4)

Уникальные значения в признаке 'education':

## Рисунок 18 - Результат выполнения программы

Задание 5. Комплексный EDA.

Датасет: heart disease (заболевания сердца).

Источник: <https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction>

Инструкции: Выполните полный цикл EDA:

1. Обзор структуры данных (.info(), .describe()).
2. Обнаружение и обработка пропущенных значений.
3. Обнаружение и удаление выбросов по признакам: age, cholesterol, restingbp, maxhr.
4. Масштабирование числовых признаков.
5. Кодирование категориальных признаков: sex, chestpain, exerciseangina, restecg.
6. Подготовьте отчет в виде Jupiter-ноутбука с комментариями к каждому этапу и промежуточными результатами.

Решение:

1. Загрузка данных. fetch\_ucirepo(id=45) загружает датасет Heart Disease из репозитория UCI. Датасет содержит медицинские показатели пациентов и диагноз (наличие/отсутствие заболевания сердца). Для удобства столбцы переименованы в понятные названия.
2. Обзор структуры данных. Метод .info() выводит информацию о типах данных и количестве непустых значений. Метод .describe() показывает статистические характеристики числовых признаков (среднее, стандартное отклонение, минимум, максимум, квартили).
3. Обнаружение и обработка пропусков. Проверяем каждый столбец на наличие пропущенных значений с помощью .isna().sum(). Пропуски заполняются: числовые признаки – медианой, категориальные – модой (наиболее частым значением).
4. Обнаружение и удаление выбросов. Используем метод межквартильного размаха (IQR). Выбросами считаются значения, выходящие за пределы интервала  $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ . Удаляем строки, содержащие выбросы по признакам Age, Cholesterol, RestingBP, MaxHR.

5. Масштабирование числовых признаков. Выполняем два типа масштабирования: стандартизация (StandardScaler) – преобразует данные к среднему 0 и стандартному отклонению 1. Нормализация (MinMaxScaler) – преобразует данные в диапазон [0, 1].

6. Кодирование категориальных признаков. В данном датасете категориальные признаки уже представлены в числовом виде (закодированы). Приводится расшифровка значений для каждого категориального признака.

7. Итоговый отчет. Сравниваются размеры датасета до и после обработки, выводятся результаты каждого этапа.

Листинг задания:

```
from ucimlrepo import fetch_ucirepo
```

```
# 1. ЗАГРУЗКА ДАННЫХ
print("\n 1. ЗАГРУЗКА ДАННЫХ")
```

```
# Загружаем датасет Heart Disease (ID = 45)
heart_disease = fetch_ucirepo(id=45)
# Данные в виде pandas DataFrame
df = heart_disease.data.original
```

```
print(f"Размер датасета: {df.shape[0]} строк, {df.shape[1]} столбцов")
print("\n Первые 5 строк:")
print(df.head())
```

```
# Переименовываем столбцы для удобства (в соответствии с описанием)
column_names = {
    'age': 'Age',
    'sex': 'Sex',
    'cp': 'ChestPainType',
    'trestbps': 'RestingBP',
    'chol': 'Cholesterol',
    'fbs': 'FastingBS',
    'restecg': 'RestingECG',
    'thalach': 'MaxHR',
    'exang': 'ExerciseAngina',
    'oldpeak': 'Oldpeak',
    'slope': 'ST_Slope',
    'ca': 'NumVessels',
    'thal': 'Thalassemia',
    'num': 'HeartDisease'
}
```

```
rename_dict = {old: new for old, new in column_names.items() if old in df.columns}
```



```

df = df.rename(columns=rename_dict)

print("\n Переименованные столбцы:")
print(df.columns.tolist())

print("ЭТАП 1. ОБЗОР СТРУКТУРЫ ДАННЫХ")

print("\n 1.1. Информация о DataFrame (info()):")
print(df.info())

print("\n 1.2. Статистическое описание числовых признаков (describe()):")
print(df.describe())

# Находим категориальные признаки (с небольшим количеством уникальных значений)
print("\n 1.3. Категориальные признаки в датасете (закодированы числами):")
for col in df.columns:
    if df[col].dtype in ['int64', 'float64'] and col != 'HeartDisease':
        unique_count = df[col].nunique()
        if unique_count <= 10:
            print(f"        {col}:    {unique_count}    уникальных    значений,    пример: {df[col].unique()[:5]}")

print("ЭТАП 2. ОБНАРУЖЕНИЕ И ОБРАБОТКА ПРОПУЩЕННЫХ ЗНАЧЕНИЙ")

missing_values = df.isna().sum()
print("\n Количество пропусков в каждом столбце:")
print(missing_values[missing_values > 0] if (missing_values > 0).any() else "Пропусков нет!")

if missing_values.sum() > 0:
    print("\n Обнаружены пропуски. Выполняем обработку...")
    for col in df.columns:
        if df[col].isna().sum() > 0:
            if df[col].dtype in ['int64', 'float64']:
                df[col].fillna(df[col].median(), inplace=True)
                print(f" {col}: заполнено медианой ({df[col].median():.2f})")
            else:
                df[col].fillna(df[col].mode()[0], inplace=True)
                print(f" {col}: заполнено модой ({df[col].mode()[0]})")
    else:
        print("\n Пропущенные значения отсутствуют!")

print("ЭТАП 3. ОБНАРУЖЕНИЕ И УДАЛЕНИЕ ВЫБРОСОВ")

outlier_features = ['Age', 'Cholesterol', 'RestingBP', 'MaxHR']
actual_features = [f for f in outlier_features if f in df.columns]

print(f"\n Анализируемые признаки: {actual_features}")

def find_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25)

```

```

Q3 = data[column].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers = data[(data[column] < lower_bound) | (data[column] > upper_bound)]
return outliers, lower_bound, upper_bound

df_original = df.copy()
df_clean = df.copy()

print("\n Удаление выбросов методом IQR:")
for feature in actual_features:
    outliers, lower, upper = find_outliers_iqr(df_clean, feature)
    print(f"\n Признак '{feature}':")
    print(f"           Q1      = {df_clean[feature].quantile(0.25):.2f},    Q3      =
{df_clean[feature].quantile(0.75):.2f}")
    print(f" Нижняя граница: {lower:.2f}, Верхняя граница: {upper:.2f}")
    print(f" Количество выбросов ДО удаления: {len(outliers)}")

    df_clean = df_clean[(df_clean[feature] >= lower) & (df_clean[feature] <= upper)]

print(f"\n Результат удаления выбросов:")
print(f" Исходное количество строк: {len(df_original)}")
print(f" После удаления выбросов: {len(df_clean)}")
print(f" Удалено строк: {len(df_original) - len(df_clean)}")
print(f" Удалено процентов: {(len(df_original) - len(df_clean)) / len(df_original) *
100:.2f}%")

df = df_clean.copy()

print("ЭТАП 4. МАСШТАБИРОВАНИЕ ЧИСЛОВЫХ ПРИЗНАКОВ")

# Определяем числовые признаки (не категориальные)
numeric_features = []
for col in df.select_dtypes(include=[np.number]).columns:
    if col != 'HeartDisease' and df[col].nunique() > 10:
        numeric_features.append(col)

print(f"\n Числовые признаки для масштабирования: {numeric_features}")

df_standard = df.copy()
df_normalized = df.copy()

if len(numeric_features) > 0:
    scaler_std = StandardScaler()
    df_standard[numeric_features] = scaler_std.fit_transform(df[numeric_features])

    scaler_mm = MinMaxScaler()
    df_normalized[numeric_features] = scaler_mm.fit_transform(df[numeric_features])

print("\n Стандартизация выполнена (среднее = 0, std = 1)")
print(" Нормализация выполнена (диапазон [0, 1])")

```

```

        if 'Age' in numeric_features:
            print(f"\nПример масштабирования для признака 'Age':")
            print(f"    Исходные:   min={df['Age'].min():.2f},   max={df['Age'].max():.2f},
mean={df['Age'].mean():.2f}")
            print(f"    Стандартизация:   min={df_standard['Age'].min():.2f},
max={df_standard['Age'].max():.2f}, mean={df_standard['Age'].mean():.2f}")
            print(f"    Нормализация:     min={df_normalized['Age'].min():.2f},
max={df_normalized['Age'].max():.2f}, mean={df_normalized['Age'].mean():.2f}")
        else:
            print("\n Нет числовых признаков для масштабирования")

print("\nЭТАП 5. КОДИРОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ")

print("\nРасшифровка категориальных признаков (UCI Heart Disease):")
print("""
Sex (пол): 1 = Male, 0 = Female
ChestPainType (cp): 1 = typical angina, 2 = atypical angina, 3 = non-anginal pain, 4 =
asymptomatic
RestingECG (restecg): 0 = normal, 1 = ST-T wave abnormality, 2 = left ventricular
hypertrophy
ExerciseAngina (exang): 1 = yes, 0 = no
ST_Slope (slope): 1 = upsloping, 2 = flat, 3 = downsloping
NumVessels (ca): number of major vessels (0-3)
Thalassemia (thal): 3 = normal, 6 = fixed defect, 7 = reversible defect
HeartDisease (num): 0 = no disease, 1-4 = disease
""")

# Категориальные признаки уже закодированы числами
cat_features = ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope',
'NumVessels', 'Thalassemia']
actual_cat_features = [f for f in cat_features if f in df.columns]

print(f"\n Категориальные признаки в датасете (уже в числовом формате):
{actual_cat_features}")

# Так как признаки уже в числовом формате, one-hot-кодирование не требуется
df_encoded = df.copy()

print("\n One-Hot Encoding не требуется (категориальные признаки уже
закодированы числами)")
print(f"\n Размер таблицы: {df.shape}")

print("\nИТОГОВЫЙ ОТЧЕТ ПО EDA")

print("\n 1. СТРУКТУРА ДАННЫХ:")
print(f"    - Исходный размер: {df_original.shape[0]} строк, {df_original.shape[1]}
столбцов")
print(f"    - После удаления выбросов: {df.shape[0]} строк")
print(f"    - После кодирования: {df_encoded.shape[1]} столбцов (кодирование не
потребовалось)")

```

```

print("\n 2. ОБРАБОТКА ПРОПУСКОВ:")
if missing_values.sum() == 0:
    print(" - Пропущенные значения: отсутствуют")
else:
    print(f" - Пропущенные значения: обработаны ({missing_values.sum()} значений)")

print("\n 3. УДАЛЕНИЕ ВЫБРОСОВ:")
print(f" - Удалено строк: {len(df_original) - len(df)}")
print(f" - Удалено процентов: {(len(df_original) - len(df)) / len(df_original) * 100:.2f}%")

print("\n 4. МАСШТАБИРОВАНИЕ:")
print(f" - Числовых признаков: {len(numeric_features)}")
print(" - Методы: StandardScaler, MinMaxScaler")

print("\n 5. КОДИРОВАНИЕ:")
print(f" - Категориальных признаков: {len(actual_cat_features)}")
print(" - Категориальные признаки уже в числовом формате (закодированы от 0/1)")
print(" - One-Hot Encoding: не требуется")

```

### Результат выполнения:

```

1. ЗАГРУЗКА ДАННЫХ
Размер датасета: 303 строк, 14 столбцов

Первые 5 строк:
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   63   1   1    145    233   1         2    150     0     2.3     3
1   67   1   4    160    286   0         2    108     1     1.5     2
2   67   1   4    120    229   0         2    129     1     2.6     2
3   37   1   3    130    250   0         0    187     0     3.5     3
4   41   0   2    130    204   0         2    172     0     1.4     1

   ca  thal  num
0  0.0  6.0   0
1  3.0  3.0   2
2  2.0  7.0   1
3  0.0  3.0   0
4  0.0  3.0   0

Переименованные столбцы:
['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS', 'RestingECG',
'MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope', 'NumVessels', 'Thalassemia', 'HeartDisease']

```

ЭТАП 1 ОБЗОР СТРУКТУРЫ ДАННЫХ

Рисунок 19 - Результат выполнения программы

### Индивидуальное практическое задание (вариант 7).

Цель: выполнить полноценный исследовательский анализ данных (EDA), применяя методы выявления и обработки пропусков, выбросов, масштабирования числовых признаков и кодирования категориальных переменных.

#### Условия к выполнению:

1. Выберите реальный табличный датасет со структурой не менее 8 признаков и целевым признаком, подходящим для задач регрессии или классификации.
2. Убедитесь, что в датасете присутствуют: числовые признаки, категориальные признаки, целевой столбец (предсказуемый признак), необязательно: пропущенные значения или выбросы.

#### Требования к выполнению:

Выполните последовательные шаги исследовательского анализа:

1. Обзор структуры данных: загрузите датасет, выведите общую информацию (`.info()`, `.describe()`), опишите сколько признаков, каких типов, какова структура целевого признака.
2. Обнаружение и обработка пропусков: определите, есть ли пропущенные значения, обоснуйте выбранный способ их устранения (удаление, заполнение средним/модой и т.д.), примените выбранный способ.
3. Обнаружение и удаление выбросов: выберите 3-5 числовых признаков, используя метод IQR, удалите выбросы, сравните объём данных до и после очистки.
4. Масштабирование числовых признаков: выполните стандартизацию (z-преобразование) с помощью `StandardScaler`, объясните, зачем выполняется масштабирование.
5. Кодирование категориальных признаков: выполните `Label Encoding` для порядковых признаков (при наличии), `One-Hot Encoding` для номинальных признаков, проверьте, исключена ли дамми-ловушка.
6. Финальный набор данных: убедитесь, что датасет не содержит пропусков, выбросов, категориальных данных в строковом виде, признаки приведены к числовому виду, масштабированы, представьте итоговый `DataFrame`, готовый к использованию в моделях.

Датасет: mpg (Fuel Economy – расход топлива автомобилей)

Источник: `seaborn.load_dataset("mpg")`

Описание датасета: данные об автомобилях 1970-1982 годов, содержащие информацию о расходе топлива, мощности, весе, ускорении и стране происхождения.

Признаки:

1. mpg – расход топлива (миль на галлон) – целевой признак (регрессия)
2. cylinders – количество цилиндров (категориальный, порядковый)
3. displacement – объем двигателя (куб. дюймы)
4. horsepower – мощность (лошадиные силы)
5. weight – вес (фунты)
6. acceleration – разгон до 60 миль/ч (секунды)
7. model\_year – год выпуска (категориальный, порядковый)
8. origin – страна происхождения (категориальный, номинативный)
9. name – название модели (удалим, так как не несет информационной ценности)

Решение:

1. Обзор структуры данных. Датасет содержит 398 строк и 9 столбцов. Целевой признак – mpg (расход топлива, миль на галлон) – является числовым, подходит для задачи регрессии. Признаки различных типов: числовые (mpg, displacement, horsepower, weight, acceleration), целочисленные категориальные (cylinders, model\_year, origin) и строковый (name). Метод `.info()` показывает типы данных и наличие пропусков. Метод `.describe()` выводит статистические характеристики числовых признаков.

2. Обнаружение и обработка пропусков. В датасете обнаружены пропуски только в признаке horsepower (6 пропусков). Столбец name удален,

так как не несет информационной ценности для анализа. Пропуски в horsepower заполнены медианным значением, так как медиана устойчива к выбросам и лучше характеризует центральную тенденцию для данных с возможными аномалиями.

3. Обнаружение и удаление выбросов. Для признаков horsepower, displacement, weight, acceleration применен метод межквартильного размаха (IQR). Выбросами считаются значения, выходящие за пределы интервала  $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ . После удаления выбросов объем данных уменьшился с 398 до 397 строк (удалена 1 строка).

4. Масштабирование числовых признаков. Выполнена стандартизация (z-преобразование) с помощью StandardScaler для признаков displacement, horsepower, weight, acceleration. Масштабирование необходимо для алгоритмов машинного обучения, чувствительных к масштабу признаков (например, SVM, KNN, линейная регрессия), чтобы признаки с большими значениями не доминировали над признаками с малыми.

5. Кодирование категориальных признаков. cylinders и model\_year – порядковые признаки (имеют естественный порядок), поэтому применен Label Encoding. origin – номинативный признак (страны: 1=USA, 2=Europe, 3=Japan), применен One-Hot Encoding с параметром drop\_first=True для предотвращения дамми-ловушки.

6. Финальный набор данных. После всех преобразований датасет не содержит пропусков, выбросов, категориальных данных в строковом виде. Все признаки приведены к числовому виду, числовые признаки масштабированы. Датасет готов к использованию в моделях машинного обучения.

Листинг задания:

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# 1. ЗАГРУЗКА ДАННЫХ И ОБЗОР СТРУКТУРЫ
print("\n 1. ЗАГРУЗКА ДАННЫХ И ОБЗОР СТРУКТУРЫ")

df = sns.load_dataset("mpg")
print(f"Размер датасета: {df.shape[0]} строк, {df.shape[1]} столбцов")
```

```

print("\n Первые 5 строк:")
print(df.head())
print("\n Информация о DataFrame:")
print(df.info())
print("\n Статистическое описание:")
print(df.describe())

print("2. ОБРАБОТКА ПРОПУСКОВ")

print("Пропуски до обработки:")
print(df.isna().sum())
df = df.drop(columns=['name'])
df['horsepower'] = df['horsepower'].fillna(df['horsepower'].median())
print(f"\n Пропуски после обработки: {df.isna().sum().sum()}")

print("3. УДАЛЕНИЕ ВЫБРОСОВ (IQR)")

outlier_features = ['horsepower', 'displacement', 'weight', 'acceleration']
df_original = df.copy()
df_clean = df.copy()

for feature in outlier_features:
    Q1 = df_clean[feature].quantile(0.25)
    Q3 = df_clean[feature].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df_clean = df_clean[(df_clean[feature] >= lower) & (df_clean[feature] <= upper)]
    print(f"{feature}: удалено {len(df_original) - len(df_clean)} выбросов")

print(f"\n Исходный размер: {len(df_original)}, После удаления: {len(df_clean)}")
df = df_clean.copy()

print("4. МАСШТАБИРОВАНИЕ ЧИСЛОВЫХ ПРИЗНАКОВ")

numeric_features = ['displacement', 'horsepower', 'weight', 'acceleration']
scaler_std = StandardScaler()
df_standard = df.copy()
df_standard[numeric_features] = scaler_std.fit_transform(df[numeric_features])
print("Стандартизация выполнена")

print("5. КОДИРОВАНИЕ КАТЕГОРИАЛЬНЫХ ПРИЗНАКОВ")

df['cylinders'] = df['cylinders'].astype('category').cat.codes
df['model_year'] = df['model_year'].astype('category').cat.codes
origin_dummies = pd.get_dummies(df['origin'], prefix='origin', drop_first=True)
df = pd.concat([df, origin_dummies], axis=1)
df = df.drop(columns=['origin'])
print("Категориальные признаки закодированы")

print("6. ФИНАЛЬНЫЙ ДАТАСЕТ")

```



```

print(f"Размер: {df.shape[0]} строк, {df.shape[1]} столбцов")
print(f"Пропуски: {df.isna().sum().sum()}")
print("\n Первые 5 строк:")
print(df.head())

```

## Результат выполнения:

```

1. ЗАГРУЗКА ДАННЫХ И ОБЗОР СТРУКТУРЫ
Размер датасета: 398 строк, 9 столбцов

Первые 5 строк:
   mpg  cylinders  displacement  horsepower  weight  acceleration  \
0  18.0          8         307.0         130.0   3504           12.0
1  15.0          8         350.0         165.0   3693           11.5
2  18.0          8         318.0         150.0   3436           11.0
3  16.0          8         304.0         150.0   3433           12.0
4  17.0          8         302.0         140.0   3449           10.5

   model_year origin          name
0          70    usa  chevrolet chevelle malibu
1          70    usa    buick skylark 320
2          70    usa  plymouth satellite
3          70    usa    amc rebel sst
4          70    usa    ford torino

Информация о DataFrame:
<class 'pandas.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   mpg             398 non-null   float64
1   cylinders        398 non-null   int64
2   displacement     398 non-null   float64
3   horsepower       392 non-null   float64
4   weight           398 non-null   int64
5   acceleration     398 non-null   float64

```

Рисунок 20 - Результат выполнения программы

**Вывод:** в ходе выполнения лабораторной работы №5 были изучены основные методы работы с библиотекой pandas для анализа и подготовки данных. В практической части освоены различные способы создания DataFrame: из словаря, из списка списков, из NumPy массива, из двух Series, а также загрузка данных из CSV-файлов и SQL-запросов. Изучены методы доступа к данным с помощью loc (по меткам) и iloc (по позициям), а также способы фильтрации строк по условиям с использованием логической индексации, методов query, isin и between. Рассмотрены операции добавления

и удаления столбцов и строк, работа с пропущенными значениями (обнаружение с помощью `isna`, заполнение с помощью `fillna`, удаление с помощью `dropna`), удаление дубликатов и сброс индекса. В рамках выполнения заданий были решены практические задачи: обработка пропусков в датасете Titanic (заполнение возраста средним значением, порта посадки – модой, удаление столбца Cabin), удаление выбросов в датасете Penguins методом межквартильного размаха IQR, масштабирование числовых признаков датасета California Housing с помощью стандартизации и нормализации, кодирование категориальных признаков датасета Adult с применением Label Encoding для порядковых признаков и One-Hot Encoding для номинативных с предотвращением дамми-ловушки. В комплексном задании EDA для датасета Heart Disease выполнен полный цикл предобработки данных, включающий обзор структуры, обработку пропусков, удаление выбросов, масштабирование и кодирование признаков. В индивидуальном задании на датасете mpg проведен самостоятельный анализ, в результате которого получен финальный набор данных, не содержащий пропусков и выбросов, с закодированными категориальными и масштабированными числовыми признаками, полностью готовый для использования в моделях машинного обучения.

## Контрольные вопросы:

1. Какие типы проблем могут возникнуть из-за пропущенных значений в данных?

Пропущенные значения могут привести к снижению размера выборки при удалении строк с пропусками, смещению статистических оценок (например, среднее значение может искажаться), невозможности работы некоторых алгоритмов машинного обучения, а также к ошибкам при визуализации и анализе данных.

2. Как с помощью методов pandas определить наличие пропущенных значений?

Для определения пропусков используются методы `.isna()` или `.isnull()`, которые возвращают булеву маску (`True` – пропуск, `False` – значение присутствует). Для подсчета количества пропусков в каждом столбце применяется `.isna().sum()`, а для проверки наличия хотя бы одного пропуска – `.isna().any().any()` или `.isna().sum().sum() > 0`.

3. Что делает метод `.dropna()` и какие параметры он принимает?

Метод `.dropna()` удаляет строки или столбцы, содержащие пропущенные значения. Основные параметры: `axis` (0 – удаление строк, 1 – удаление столбцов), `how` ('any' – удалять если есть хотя бы один пропуск, 'all' – удалять только если все значения пропущены), `thresh` (минимальное количество непустых значений для сохранения строки/столбца), `subset` (список столбцов для проверки пропусков), `inplace` (изменять исходный DataFrame или вернуть копию).

4. Чем различаются подходы заполнения пропусков средним, медианой и модой?

Среднее арифметическое чувствительно к выбросам и подходит для симметричных распределений без аномалий. Медиана устойчива к выбросам

и рекомендуется для асимметричных распределений. Мода (наиболее частое значение) применяется для категориальных признаков или для числовых с повторяющимися значениями.

5. Как работает метод `.fillna(method='ffill')` и в каких случаях он применим?

Метод `fillna(method='ffill')` (forward fill) заполняет пропуски предыдущим (сверху) непустым значением. Это полезно для временных рядов, где данные последовательны и пропуски можно заполнить предыдущим наблюдением. Аналогично существует метод `'bfill'` (backward fill), заполняющий следующим значением.

6. Какую задачу решает метод `.interpolate()` и чем он отличается от `.fillna()`?

Метод `.interpolate()` выполняет интерполяцию — вычисляет пропущенные значения на основе соседних (линейная, полиномиальная и другие виды интерполяции). В отличие от `.fillna()`, который просто копирует существующие значения, `.interpolate()` рассчитывает новые значения, учитывая тенденцию изменения данных.

7. Что такое выбросы и почему они могут исказить результаты анализа?

Выбросы — это аномальные наблюдения, значительно отличающиеся от основной массы данных. Они могут исказить статистические характеристики (среднее, стандартное отклонение), влиять на корреляции, смещать коэффициенты моделей машинного обучения и приводить к неверным выводам при визуализации.

8. В чём суть метода межквартильного размаха (IQR) и как он используется для обнаружения выбросов?

IQR (Interquartile Range) – это разница между третьим (Q3) и первым (Q1) квартилями:  $IQR = Q3 - Q1$ . Выбросами считаются значения, выходящие за пределы интервала  $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ . Множитель 1.5 является эмпирическим правилом Тьюки.

9. Как вычислить границы IQR и применить их в фильтрации?

Границы вычисляются по формулам: нижняя =  $Q1 - 1.5 \times IQR$ , верхняя =  $Q3 + 1.5 \times IQR$ . Фильтрация выполняется с помощью условия: `df[(df[column] >= lower_bound) & (df[column] <= upper_bound)]`.

10. Что делает метод `.clip()` и как его можно использовать для обработки выбросов?

Метод `.clip(lower, upper)` ограничивает значения признака заданными границами: все значения ниже `lower` заменяются на `lower`, выше `upper` – на `upper`. Это позволяет не удалять выбросы, а "прижимать" их к границам, сохраняя объем данных.

11. Зачем может потребоваться логарифмическое преобразование числовых признаков?

Логарифмическое преобразование применяется для нормализации сильно скошенных (асимметричных) распределений, приближения их к нормальному, а также для стабилизации дисперсии и уменьшения влияния выбросов.

12. Какие графические методы позволяют обнаружить выбросы (указать не менее двух)?

Для обнаружения выбросов используются: `boxplot` (ящик с усами – точки за пределами усов являются выбросами), `scatter plot` (точечная диаграмма – аномальные точки видны отдельно), гистограмма (хвосты распределения), а также Z-score графики и Q-Q plots.

13. Почему важно быть осторожным при удалении выбросов из обучающих данных?

Удаление выбросов может привести к потере важной информации, если выбросы являются аномалиями, представляющими интерес (например, мошеннические транзакции). Также чрезмерное удаление сокращает объем обучающей выборки, что может ухудшить обобщающую способность модели.

14. Зачем необходимо масштабирование признаков перед обучением моделей?

Масштабирование необходимо, чтобы признаки с большими диапазонами значений не доминировали над признаками с малыми диапазонами. Алгоритмы, чувствительные к масштабу (например, SVM, KNN, линейная регрессия с регуляризацией, градиентный спуск), работают корректно только после масштабирования.

15. Чем отличается стандартизация от нормализации?

Стандартизация (StandardScaler) преобразует данные так, чтобы они имели среднее 0 и стандартное отклонение 1:  $z = (x - \mu) / \sigma$ . Нормализация (MinMaxScaler) сжимает данные в заданный диапазон, обычно  $[0, 1]$ :  $x' = (x - \min) / (\max - \min)$ .

16. Что делает StandardScaler и как рассчитываются преобразованные значения?

StandardScaler вычисляет для каждого признака среднее ( $\mu$ ) и стандартное отклонение ( $\sigma$ ), затем преобразует каждое значение по формуле:  $z = (x - \mu) / \sigma$ . В результате преобразованные значения имеют среднее 0 и стандартное отклонение 1.

17. Как работает MinMaxScaler и когда его использование предпочтительно?

MinMaxScaler преобразует данные в заданный диапазон (по умолчанию  $[0, 1]$ ) по формуле:  $x' = (x - \min) / (\max - \min)$ . Предпочтителен, когда данные имеют известные границы (например, проценты, яркость пикселей) или когда требуется сохранить нулевые значения.

18. В чём преимущества RobustScaler при наличии выбросов?

RobustScaler использует медиану и межквартильный размах (IQR) вместо среднего и стандартного отклонения. Это делает его устойчивым к выбросам, так как медиана и квартили менее чувствительны к аномалиям, чем среднее и дисперсия.

19. Как реализовать стандартизацию с помощью `.mean()` и `.std()` вручную в pandas?

Стандартизацию можно выполнить вручную:  $df\_std = (df - df.mean()) / df.std()$ . Для каждого признака отдельно:  $df[col] = (df[col] - df[col].mean()) / df[col].std()$ .

20. Какие типы моделей наиболее чувствительны к масштабу признаков?

Наиболее чувствительны к масштабу признаков модели, основанные на расстояниях (KNN, K-средних), модели с регуляризацией (Lasso, Ridge, ElasticNet), метод опорных векторов (SVM), нейронные сети и градиентный спуск. Деревья решений и ансамбли (Random Forest, Gradient Boosting) менее чувствительны к масштабу.

21. Почему необходимо преобразовывать категориальные признаки перед обучением модели?

Большинство алгоритмов машинного обучения работают только с числовыми данными. Категориальные признаки в виде строк необходимо преобразовать в числа с помощью кодирования (Label Encoding, One-Hot Encoding и др.), чтобы модель могла их обрабатывать.

22. Что такое порядковый признак? Приведите пример.

Порядковый (ординальный) признак – это категориальный признак, категории которого имеют естественный порядок или иерархию. Примеры: уровень образования (начальное, среднее, высшее), оценка (отлично, хорошо, удовлетворительно), размер (маленький, средний, большой).

23. Что такое номинальный признак? Приведите пример.

Номинативный признак – это категориальный признак, категории которого не имеют естественного порядка. Примеры: цвет (красный, синий, зеленый), страна происхождения (Россия, США, Германия), тип оплаты (наличные, карта, перевод).

24. Как работает метод `.factorize()` и для каких случаев он подходит?

Метод `.factorize()` преобразует категориальные значения в числовые коды, присваивая каждой уникальной категории целое число (начиная с 0). Возвращает массив кодов и массив уникальных значений. Подходит для быстрого кодирования без учета порядка категорий.

25. Как применить метод `.map()` для кодирования категориальных признаков с известным порядком?

Метод `.map()` применяется с предварительно созданным словарем соответствий категория – число. Например: `mapping = {'low': 1, 'medium': 2, 'high': 3}`; `df['feature_encoded'] = df['feature'].map(mapping)`. Это позволяет задать явный порядок категорий.



26. Что делает класс `OrdinalEncoder` из `scikit-learn`?

`OrdinalEncoder` преобразует категориальные признаки в числовые коды (0, 1, 2, ...). В отличие от `LabelEncoder` (работает с одним признаком), `OrdinalEncoder` может одновременно кодировать несколько категориальных признаков, а также позволяет задать порядок категорий через параметр `categories`.

27. В чём суть `one-hot` кодирования и когда оно применяется?

`One-Hot Encoding` создает для каждой категории отдельный бинарный столбец: 1 – если категория присутствует, 0 – если отсутствует. Применяется для номинативных признаков, где порядок категорий не важен, чтобы избежать ложной порядковой зависимости.

28. Как избежать дамми-ловушки при `one-hot` кодировании?

Дамми-ловушка – это мультиколлинеарность, возникающая, когда все бинарные столбцы суммируются в единицу. Для ее предотвращения используется параметр `drop_first=True` (в `pd.get_dummies`) или `OneHotEncoder(drop='first')`, который удаляет один из бинарных столбцов.

29. Как работает `OneHotEncoder` из `scikit-learn` и чем он отличается от `pd.get_dummies()`?

`OneHotEncoder` преобразует категориальные признаки в разреженную матрицу, работает с числовыми категориями и может применяться в пайплайнах. `pd.get_dummies()` проще, работает непосредственно с `DataFrame`, создает полноценные столбцы, но не интегрируется в пайплайны `scikit-learn`.

30. В чём суть метода `target encoding` и какие риски он в себе несёт?

`Target encoding` заменяет категорию на среднее значение целевого признака для этой категории. Это позволяет учесть взаимосвязь категории с целевой переменной. Риски: переобучение (особенно для редких категорий),

утечка данных (если вычислять на всей выборке), смещение оценок. Для снижения рисков используют сглаживание (smoothing) и кросс-валидацию.